

Tool calling con l'omni: come avviene, cosa rallenta, come migliorarlo

Ramo HUD — MiniCPM-o 4.5 (omni) + modello separato per il tool calling + schermo dell'operatore. Alessandro, 03/09/2026.

1. Come avviene ora

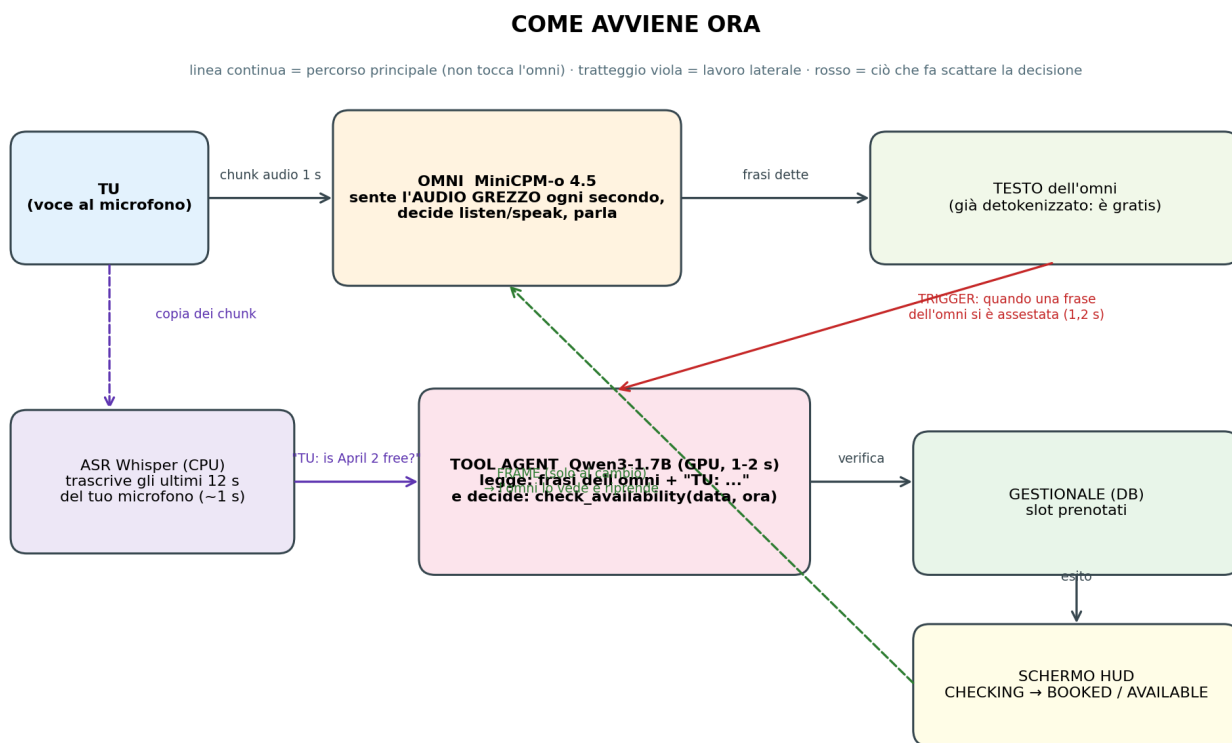


Figura 1. L'omni riceve il tuo audio grezzo e parla; il suo testo è già detokenizzato e non costa nulla. Il tuo testo invece non esiste (end-to-end): lo produce un ASR laterale. Il tool agent legge entrambi e decide la chiamata; il risultato torna all'omni come cambio dello schermo (frame).

- **Trigger:** la decisione parte quando una **frase dell'omni** si è assestata (1,2 s senza nuovi delta), non sulle tue parole.
- **Ingresso della decisione:** le frasi dell'omni + la trascrizione degli **ultimi 12 s del tuo microfono** (Whisper su CPU, ~1 s).
- **Uscita:** `check_availability(data, ora)` → gestionale → schermo HUD → frame all'omni, che lo legge e riprende da solo (misurato: 0,4-0,7 s dopo il frame).

2. Cosa stiamo rallentando

COSA STIAMO RALLENTANDO (contesa di risorse, non percorso)

GPU (32,6 GB)
omni 23,6 GB + tool agent 3,5 GB
l'omni deve decodificare 1 unità/s: quando
Qwen genera (1-2 s) la GPU è condivisa → singhiozzo

CPU (24 core)
Whisper small ~1 s per chiamata, pochi core;
l'omni usa la CPU solo per orchestrare:
contesa trascurabile

MISURATO sulle tue sessioni (registrazioni del gateway):
cadenza dell'omni 1,00 s per unità (mediana) · chunk del mic ogni 1,00 s · 2-4 singhiozzi di 1,5-1,8 s per sessione,
nei momenti in cui lavora il tool agent → nessun rallentamento sistematico, qualche mezzo secondo perso ogni tanto

Niente sta **sul percorso** dell'omni: non aspetta la trascrizione né il tool agent. La sola interferenza è la **contesa di risorse**: il tool agent usa la stessa GPU per 1-2 s a ogni chiamata; Whisper usa un po' di CPU. Effetto misurato: 2-4 singhiozzi di 1,5-1,8 s per sessione, cadenza altrimenti a 1,00 s.

Togliere l'ASR? Si può, ma allora il tool agent vedrebbe solo le frasi dell'omni, che spesso non ripete data e ora («Let me check.» e basta): è il caso dei 30 secondi di silenzio visti nei test. Il tuo testo serve; va reso più leggero, non eliminato.

3. Perché oggi sbaglia (falsi positivi)

- Vede **una frase alla volta**, senza sapere chi ha introdotto la data: ha preso per richieste una domanda dell'operatore («...for today or tomorrow evening?») e un annuncio di esito («March 31 at 3 pm is available»).
- Non ha memoria delle verifiche già fatte → duplicati; e il buffer di 12 s mescola la tua domanda vecchia con le parole nuove.

4. Proposta

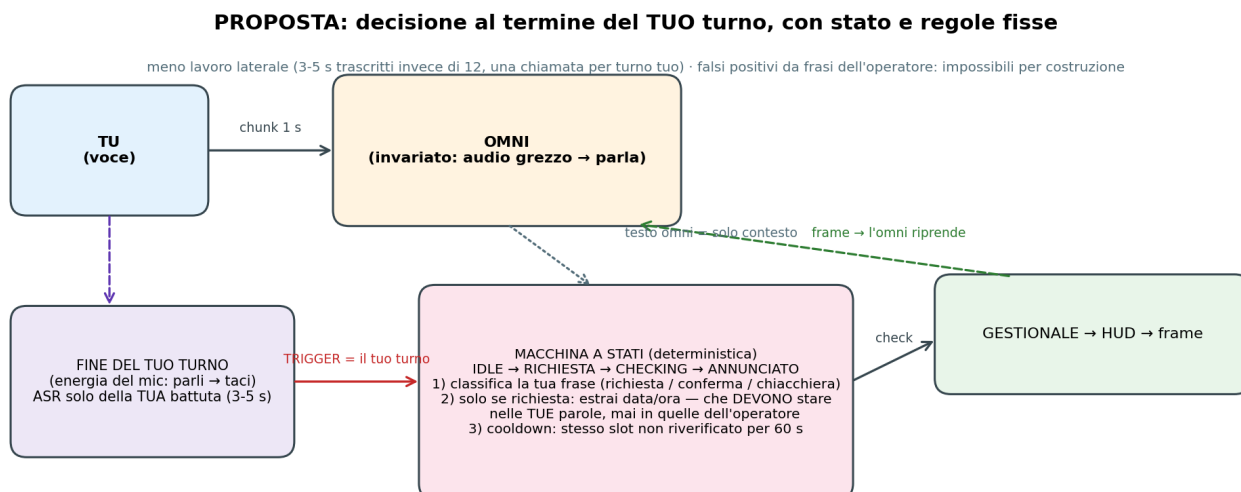


Figura 2. La decisione parte al termine del tuo turno; una macchina a stati deterministica governa quando consultare il modello; il modello fa due compiti stretti (classificare, poi estrarre) invece di una chiamata aperta; la data deve stare nelle tue parole.

- **Trigger = il tuo turno** (energia del microfono: parli → taci): si trascrive solo la tua battuta (3-5 s) e si decide una volta per turno tuo. Meno lavoro laterale, meno contesa.
- **Macchina a stati**: IDLE → RICHIESTA → CHECKING → ANNUNCIATO; il testo dell'omni entra solo come contesto; cooldown di 60 s sullo stesso slot; nessuna consultazione nel turno subito dopo un frame di esito.
- **Due passi**: classificazione in una parola (richiesta / conferma / chiacchiera / correzione) e, solo se richiesta, estrazione di data e ora con vincolo «devono comparire nelle parole dell'utente».
- **Contesto vero**: ultimi 4-6 turni con ruolo e ordine + stato dello schermo («ultima verifica: March 31 15:00 → BOOKED, 12 s fa») + una reason di una riga nel registro.
- **Ultima leva**, solo se serve: sidecar più grande (Qwen3-4B quantizzato) — costa latenza.

Predizione: falsi positivi da ~3 per sessione a ~0; latenza della decisione uguale o minore. Il «Let me check» seguito da 30 s di silenzio è invece un comportamento dell'omni e va indagato a parte.